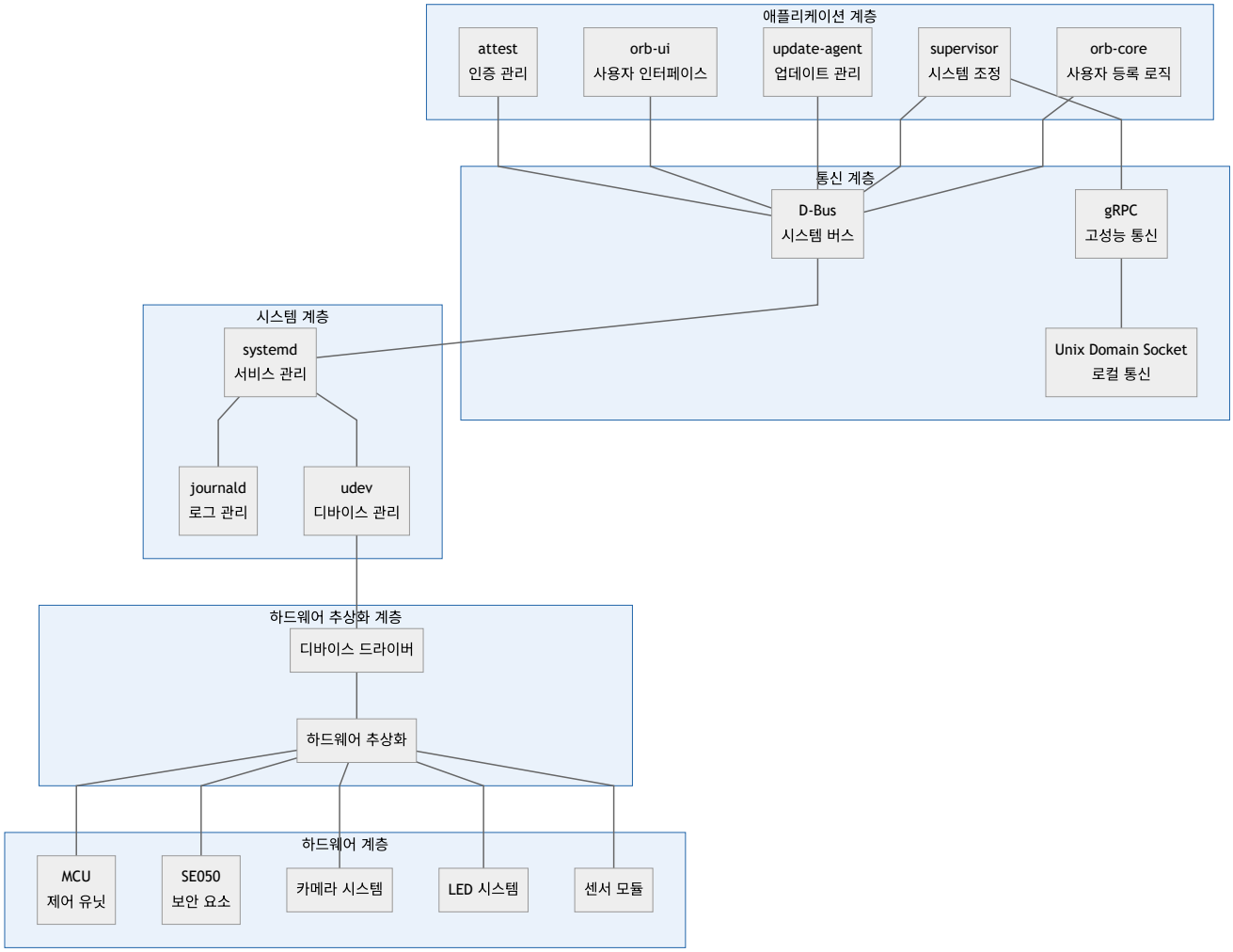
# Orb 시스템 아키텍처

## 개요

Orb 시스템은 생체인식 기반 신원 확인을 위한 분산 아키텍처로 설계되었습니다. 여러 독립적인 컴포넌트들이 표준화된 통신 프로토콜을 통해 협력하여 안전하고 신뢰할 수 있는 World ID 검증 서비스를 제공합니다.

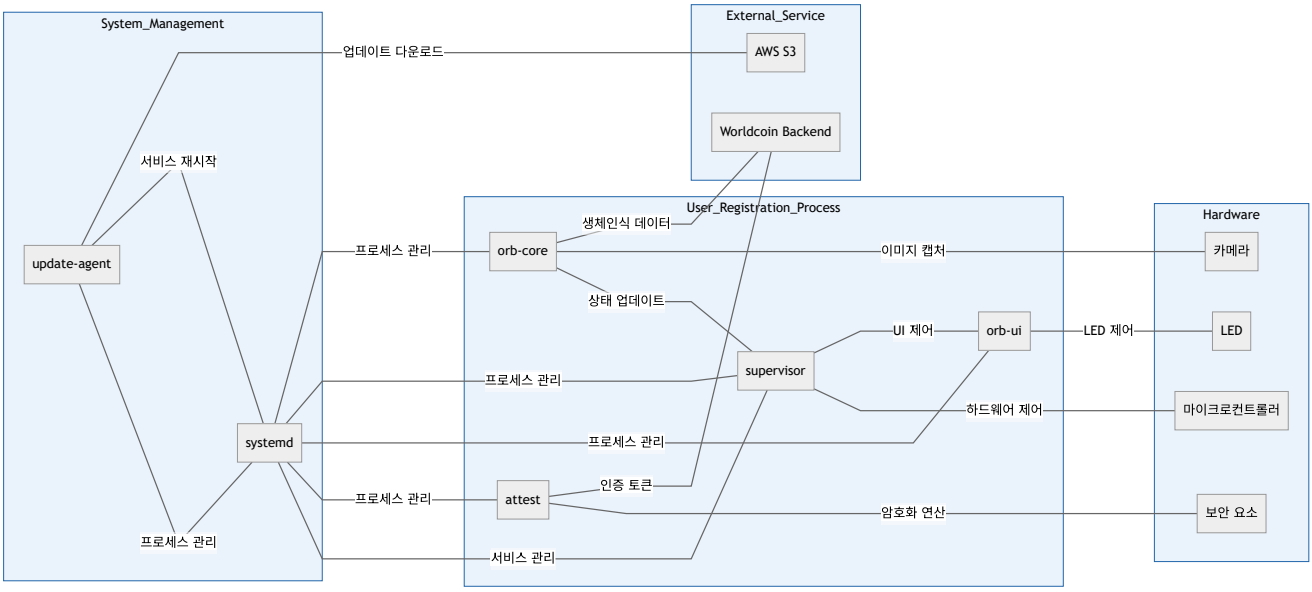
## 전체 시스템 아키텍처

### 계층별 구조

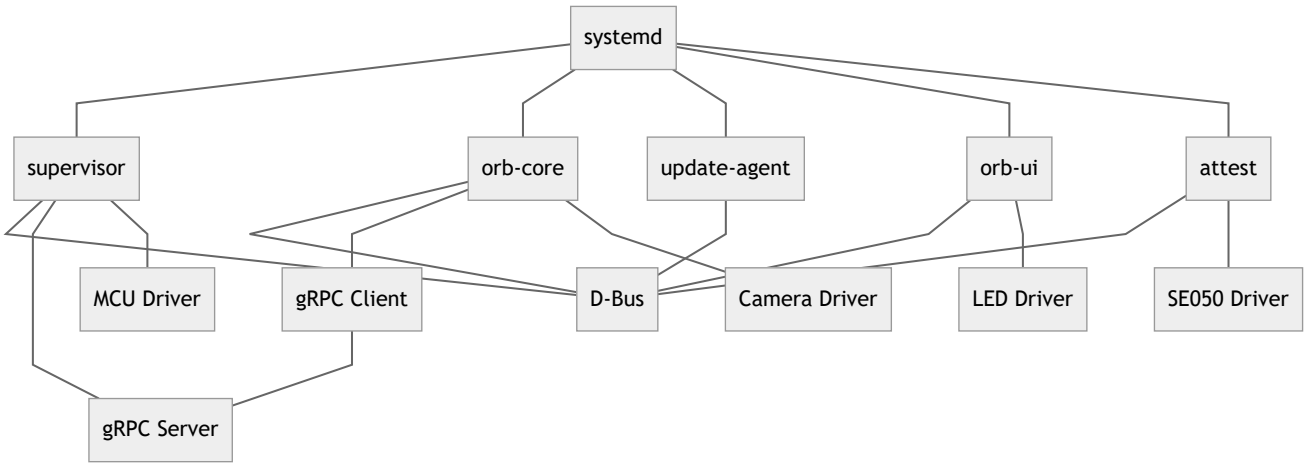

애플리케이션 계층
attest
인증 관리
orb-ui
사용자 인터페이스
update-agent
업데이트 관리
supervisor
시스템 조정
orb-core
사용자 등록 로직
통신 계층
D-Bus
시스템 버스
gRPC
고성능 통신
Unix Domain Socket
로컬 통신
시스템 계층
systemd
서비스 관리
journald
로그 관리
udev
디바이스 관리
하드웨어 추상화 계층
디바이스 드라이버
하드웨어 추상화
하드웨어 계층
MCU
제어 유닛
SE050
보안 요소
카메라 시스템
LED 시스템
센서 모듈

# 컴포넌트 간 관계도

## 핵심 컴포넌트 상호작용

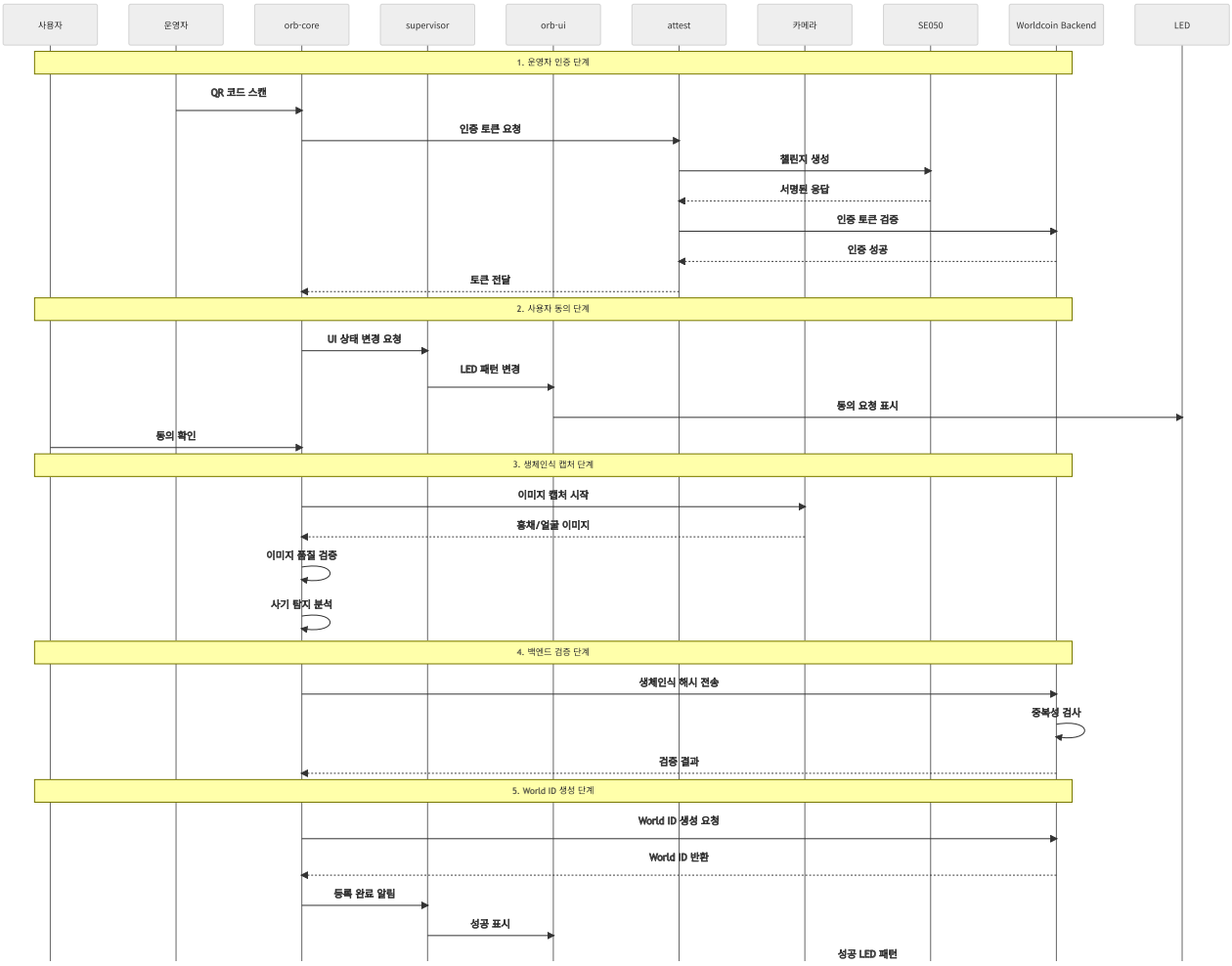



## 의존성 관계

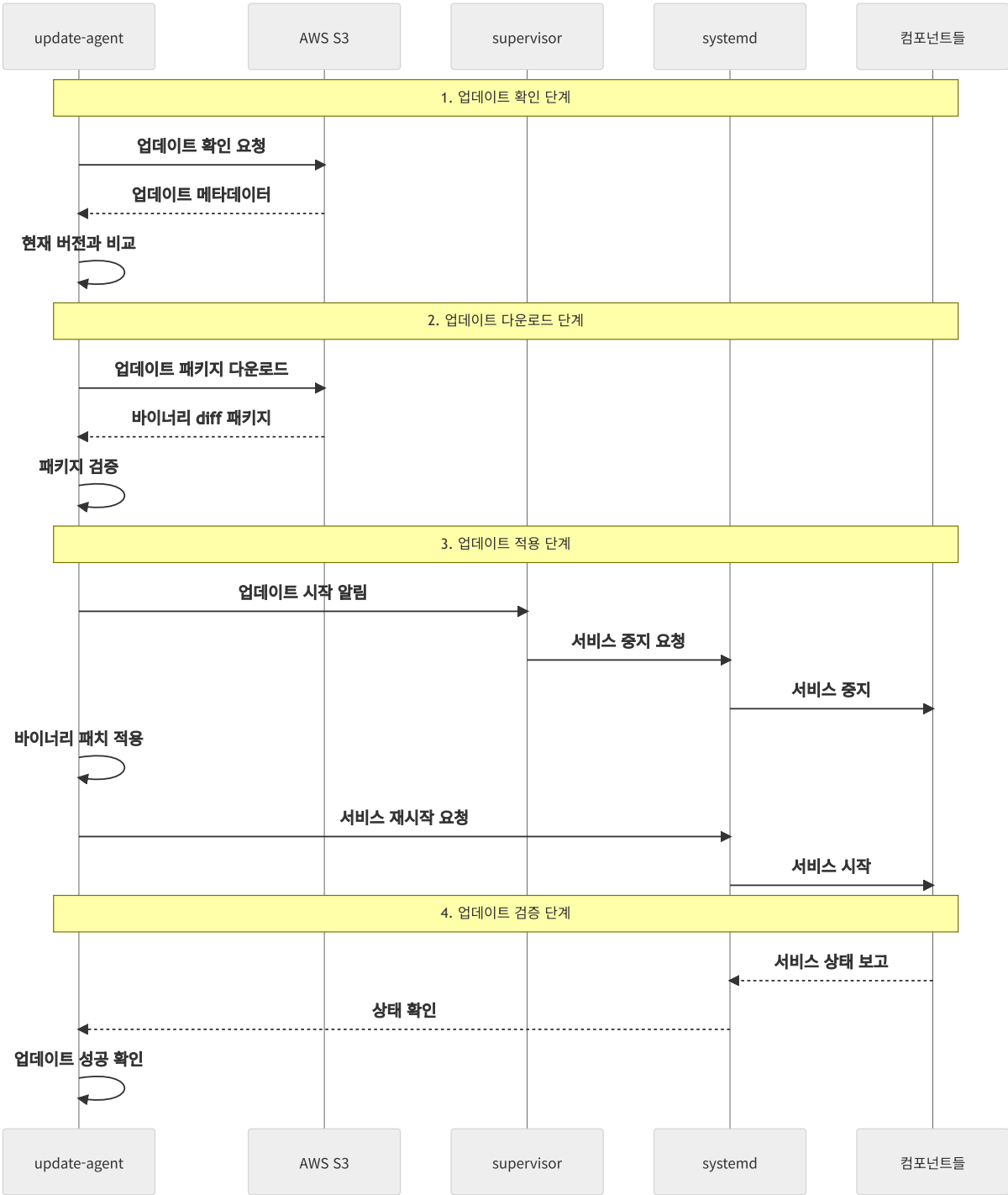



# 데이터 흐름 다이어그램

## 사용자 등록 프로세스 데이터 흐름


사용자
운영자
orb-core
supervisor
orb-ui
attest
카메라
SE050
Worldcoin Backend
LED
1. 운영자 인증 단계
QR 코드 스캔
인증 토큰 요청
챌린지 생성
서명된 응답
인증 토큰 검증
인증 성공
토큰 전달
2. 사용자 동의 단계
UI 상태 변경 요청
LED 패턴 변경
동의 요청 표시
동의 확인
3. 생체인식 캡처 단계
이미지 캡처 시작
홍채/얼굴 이미지
이미지 품질 검증
사기 탐지 분석
4. 백엔드 검증 단계
생체인식 해시 전송
중복성 검사
검증 결과
5. World ID 생성 단계
World ID 생성 요청
World ID 반환
등록 완료 알림
성공 표시
성공 LED 패턴

# 시스템 업데이트 데이터 흐름


update-agent
AWS S3
supervisor
systemd
컴포넌트들
1. 업데이트 확인 단계
업데이트 확인 요청
업데이트 메타데이터
현재 버전과 비교
2. 업데이트 다운로드 단계
업데이트 패키지 다운로드
바이너리 diff 패키지
패키지 검증
3. 업데이트 적용 단계
업데이트 시작 알림
서비스 중지 요청
서비스 중지
바이너리 패치 적용
서비스 재시작 요청
서비스 시작
4. 업데이트 검증 단계
서비스 상태 보고
상태 확인
업데이트 성공 확인
update-agent
AWS S3
supervisor
systemd
컴포넌트들

# 통신 프로토콜 개요

## D-Bus 통신

D-Bus는 Orb 시스템의 주요 IPC 메커니즘으로 사용됩니다.

**주요 특징:**

- **시스템 버스**: 시스템 전체 서비스 간 통신
- **세션 버스**: 사용자 세션 내 통신 (사용하지 않음)
- **메시지 타입**: 메서드 호출, 신호, 오류 메시지

**서비스 인터페이스:**

```
// D-Bus 인터페이스 예시
pub trait OrbCoreInterface {
fn start_signup(&self, operator_qr: String) -> Result<String, Error>;
fn get_status(&self) -> Result<SystemStatus, Error>;
fn cancel_signup(&self) -> Result<(), Error>;
}

pub trait SupervisorInterface {
fn set_ui_state(&self, state: UiState) -> Result<(), Error>;
fn get_system_health(&self) -> Result<HealthStatus, Error>;
```

```
fn shutdown_system(&self) -> Result<(), Error>;
}
```

**신호 브로드캐스팅:**

```
// 시스템 이벤트 신호
pub enum SystemSignal {
SignupStarted { session_id: String },
SignupCompleted { world_id: String },
SignupFailed { error: String },
UpdateAvailable { version: String },
SystemError { component: String, error: String },
}
```

## gRPC 통신

gRPC는 supervisor와 다른 컴포넌트 간의 고성능 통신에 사용됩니다.

**프로토콜 정의:**

```
// supervisor.proto
service SupervisorService {
rpc GetSystemStatus(Empty) returns (SystemStatus);
rpc SetHardwareState(HardwareState) returns (Empty);
rpc StreamEvents(Empty) returns (stream SystemEvent);
}
```

```
message SystemStatus {
string version = 1;
repeated ComponentStatus components = 2;
HardwareStatus hardware = 3;
}

message ComponentStatus {
string name = 1;
bool is_running = 2;
string version = 3;
repeated string errors = 4;
}
```

**Unix Domain Socket 사용:**

- 경로: `/var/run/orb/supervisor.sock`
- 권한: 특정 그룹만 접근 가능
- 성능: TCP보다 빠른 로컬 통신

## 하드웨어 통신 프로토콜

### SE050 통신

- **인터페이스**: I2C
- **프로토콜**: APDU (Application Protocol Data Unit)
- **보안**: 하드웨어 기반 암호화

**MCU 통신**

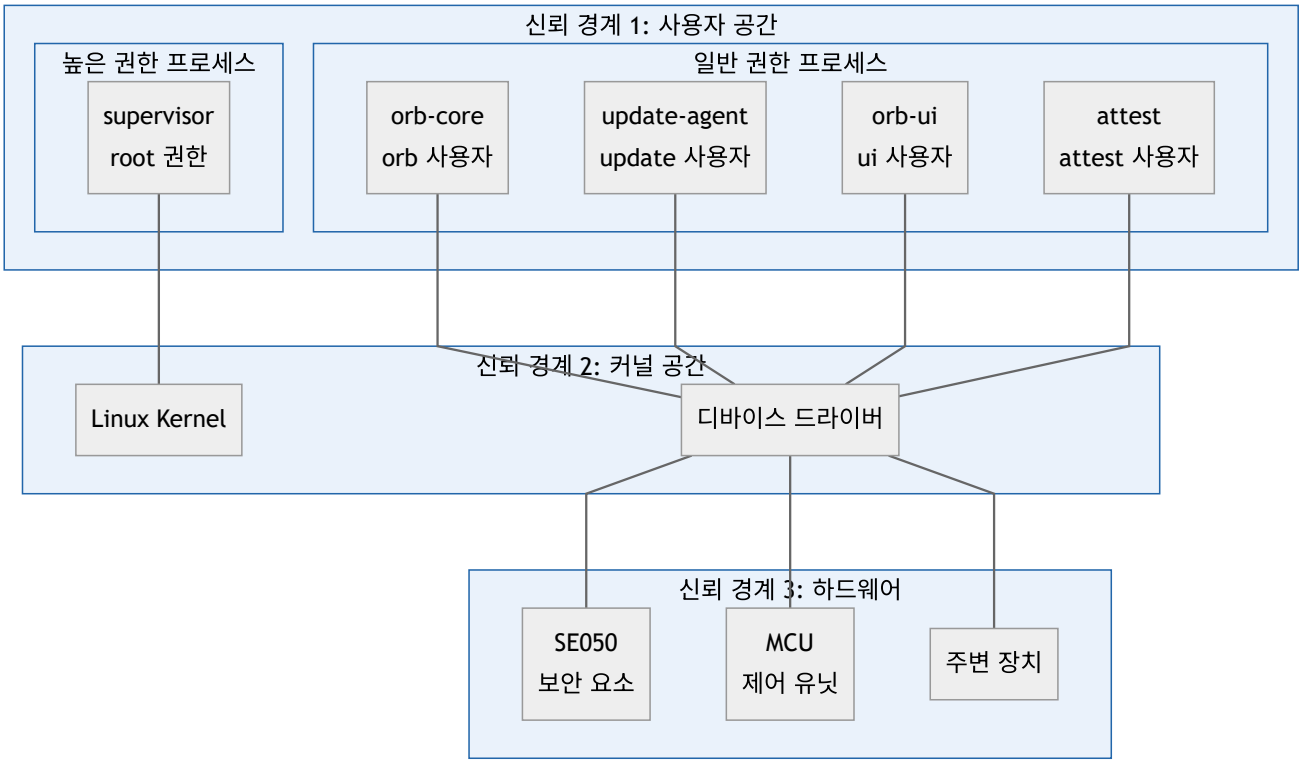
- **인터페이스**: CAN 버스, UART
- **프로토콜**: 커스텀 바이너리 프로토콜
- **기능**: 센서 데이터, 액추에이터 제어

**카메라 시스템**

- **인터페이스**: USB, MIPI CSI
- **프레임워크**: GStreamer 파이프라인
- **형식**: H.264, MJPEG

## 시스템 경계 및 인터페이스

### 내부 시스템 경계




## 외부 인터페이스

### 네트워크 인터페이스

**Worldcoin Backend API:**

- **프로토콜**: HTTPS/TLS 1.3
- **인증**: mTLS (상호 TLS 인증)
- **엔드포인트**:
- `/api/v1/signup` - 사용자 등록
- `/api/v1/verify` - 생체인식 검증
- `/api/v1/attest` - 디바이스 인증

**AWS S3 업데이트 서비스:**

- **프로토콜**: HTTPS
- **인증**: AWS Signature V4
- **버킷**: `orb-updates-{region}`

**물리적 인터페이스**

**사용자 인터페이스:**

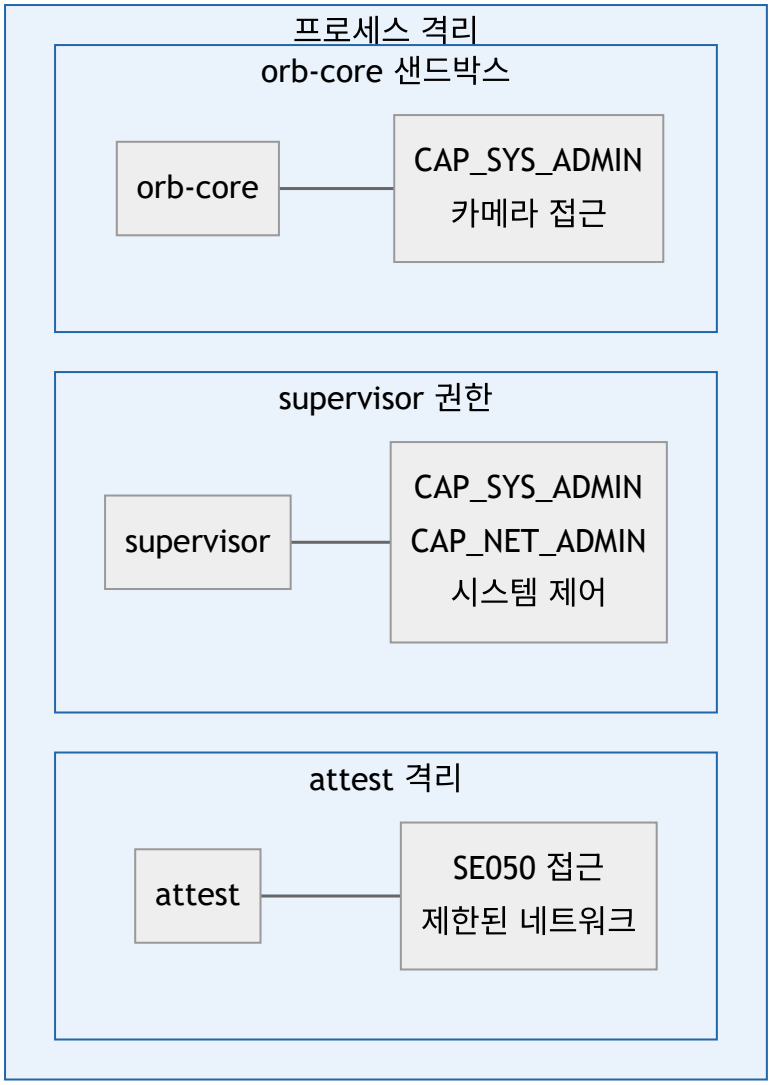
- **LED 링**: 60fps RGB 제어
- **스피커**: 오디오 피드백
- **카메라**: 홍채/얼굴 인식

**운영자 인터페이스:**

- **QR 스캐너**: 운영자 인증
- **터치 인터페이스**: 시스템 제어 (선택사항)

## 보안 경계

**프로세스 격리**




**데이터 흐름 제어**

**입력 검증:**

- 모든 외부 입력에 대한 엄격한 검증
- 프로토콜 버퍼를 통한 타입 안전성
- 크기 제한 및 형식 검증

**출력 필터링:**

- 민감한 정보 로그 제외
- 생체인식 데이터 해시화
- 오류 메시지 정보 제한

# 확장성 및 유지보수성

## 모듈화 설계

각 컴포넌트는 독립적으로 개발, 테스트, 배포할 수 있도록 설계되었습니다.

**인터페이스 표준화:**

- D-Bus 인터페이스 정의
- gRPC 서비스 계약
- 하드웨어 추상화 계층

**버전 관리:**

- 컴포넌트별 독립적 버전 관리
- 하위 호환성 보장
- 점진적 업그레이드 지원

## 모니터링 및 관찰성

**로그 수집:**

- 중앙화된 journald 로그
- 구조화된 로그 형식 (JSON)
- 로그 레벨별 분류

**메트릭 수집:**

- 시스템 성능 메트릭
- 비즈니스 메트릭 (등록 성공률 등)
- 하드웨어 상태 메트릭

**분산 추적:**

- 요청별 추적 ID
- 컴포넌트 간 호출 추적
- 성능 병목 지점 식별

## 결론

Orb 시스템의 아키텍처는 보안, 성능, 유지보수성을 균형있게 고려하여 설계되었습니다. 모듈화된 컴포넌트 구조와 표준화된 통신 프로토콜을 통해 복잡한 생체인식 시스템을 안정적으로 운영할 수 있으며, 지속적인 개선과 확장이 가능한 유연한 아키텍처를 제공합니다.